

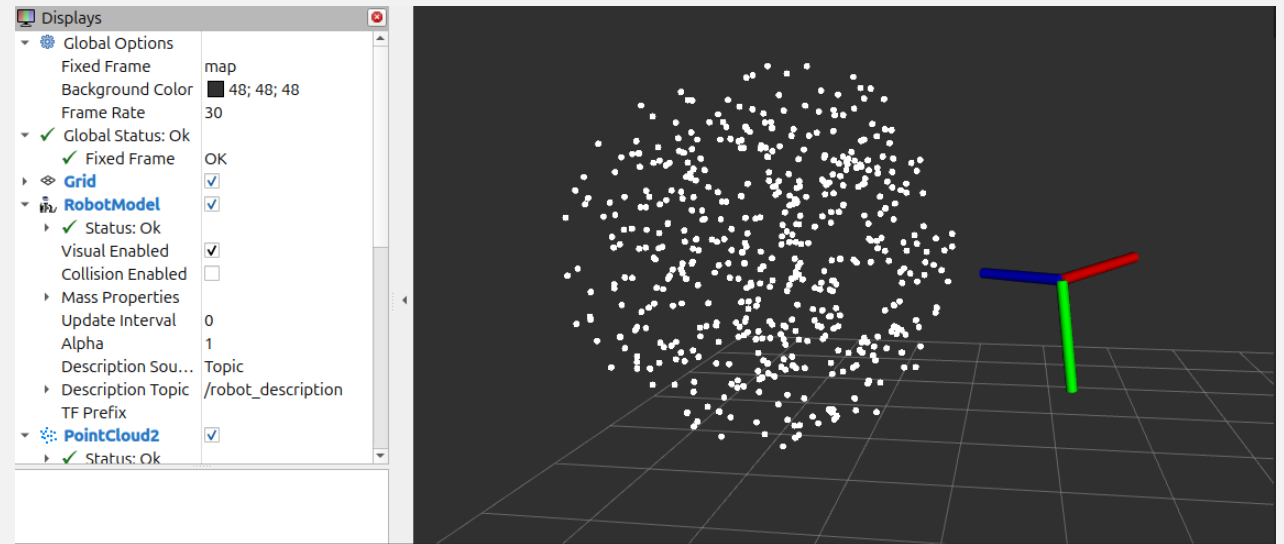
AULA 9

Navegação e Nav2

2(½) EXERCÍCIO DA "ÚLTIMA" AULA

Vasculhe a nuvem de pontos (abordagem livre, sejam criativos) e eliminem todos os pontos que não estejam em uma banda de até 0.5m da altura do frame **drone_frame** em relação à **map**.

Após isso publiquem a nuvem de pontos resultante em um tópico do tipo **sensor_msgs/msg/PointCloud2**.



NAVEGAÇÃO

- Definição : Conjunto de ações capazes de permitir que um robô se mova de um ponto A até um ponto B de forma autônoma, levando em conta obstáculos, segurança e restrições físicas.
- Para isso, o robô precisa perceber o ambiente, localizar-se nesse ambiente, planejar um caminho e controlar seus movimentos para seguir esse caminho com robustez
 - Ou seja: Auto localização + Planejamento e execução de rotas
- Ações típicas de um sistema de navegação
 - Localização
 - Mapeamento
 - Planejamento
 - Rotinas de recuperação
 - Comandos de movimento

TIPOS DE NAVEGAÇÃO

- O algoritmo de navegação, em larga escala, é baseado no tipo de sensores que o robô possui
- **Dead Reckoning** : Por odometria ou sensores inerciais, a posição é estimada pela soma de movimentos (~ malha aberta)
- **Navegação Reativa** : Robô não possui um mapa a priori do ambiente, apenas reage a leitura momentânea de seus sensores, sem uma memória
 - Comum em robôs aspiradores baratos, robôs sumo ou seguidores de linha.
 - Frágil à ficar preso e não seguirá uma rota ótima
- **SLAM** (*Simultaneous Localization and Mapping*) : O robô constrói um mapa enquanto navega e usa esse mapa para saber onde está
 - Lidar (Laser SLAM) ou visual SLAM (vSLAM)
- **Métodos Globais** : Utilização de GNSS (ex: GNSS americano = GPS)
- **Navegação por referências/marcos** : Referências fixas no ambiente



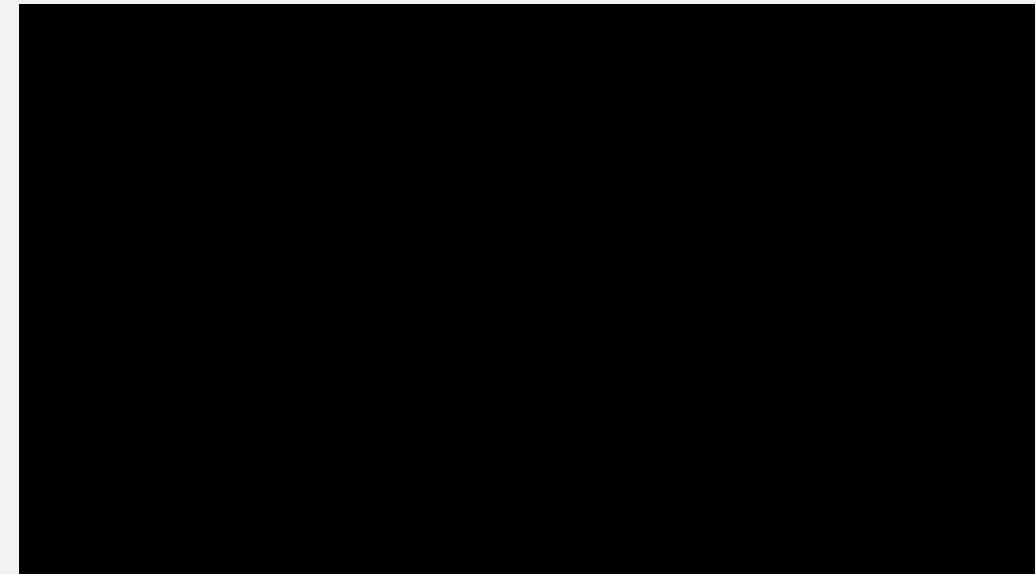
NAV2

NAVEGAÇÃO NO ROS2



[Nav2 — Nav2 1.0.0 documentation](#)

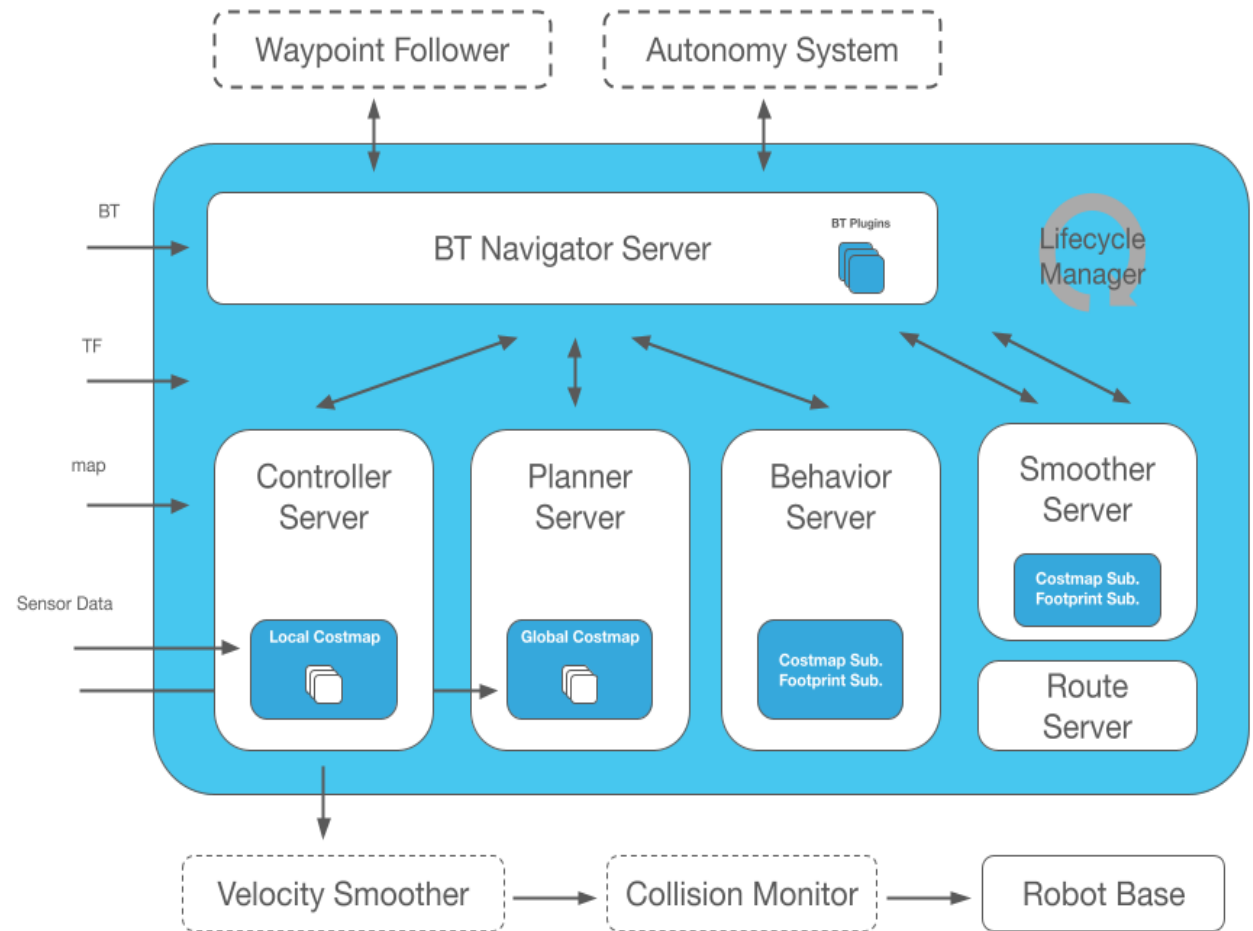
- Pacote de navegação padrão do ROS2 é o **Nav2**, sucessor espiritual do Navigation Stack do ROS 1
 - Pacotes similares :
 - Autoware : Carros Autônomos
 - Aerostack2 : Drones/Aéreos (NAV2 é fundamentalmente 2D (projeta o mundo em um mapa plano))
 - MoveIt 2 : Manipuladores e Robôs Móveis Complexos
 - ...
 - O pacote Nav2 é baseado em ações em conjunto com árvores de tomada de decisão (Behavior Trees (BT)), altamente customizável através de interfaces padrões e plugins
- [Navigation Concepts — Nav2 1.0.0 documentation](#)



Nav2 sendo usado para navegação autônoma de um robô no mundo real

NAVEGAÇÃO NO ROS2

- Cada server especializado é fortemente modularizado
- É possível ter mais de um por categoria
- A BT Server orchestra o chamamento dos servers (implementados com ações)
- Todos os nós que lançam essas ações são *lifecycle nodes*
 - Nós com uma máquina de estados interna



NAV2: CONCEITOS

Estrutura do Nav2

- Usa árvores de comportamento para chamar servidores de ação
 - Cada servidor faz uma tarefa distinta: planejar caminhos para o robô, suavizar os caminhos, gerar referências para controladores, etc.
 - Cada ação é providenciada por um nó separado
 - Cada servidor pode carregar múltiplos *plugins*, para ter comportamentos diferentes de acordo com o contexto
- Entradas esperadas do Nav2:
 1. Behavior Tree em arquivo .xml
 2. Transformações \tf
 3. Mapa de custo do ambiente
 4. Dados de sensores relevantes

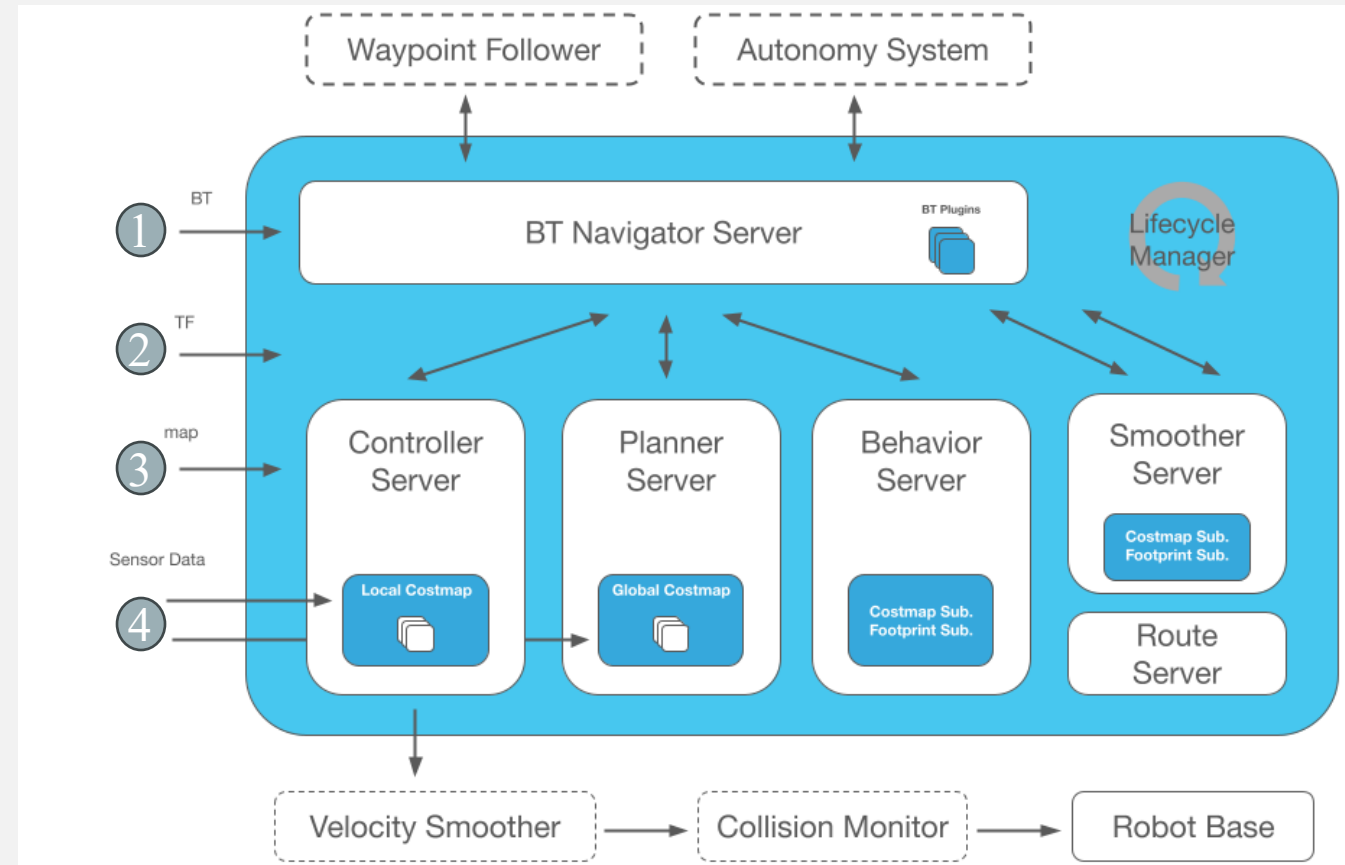
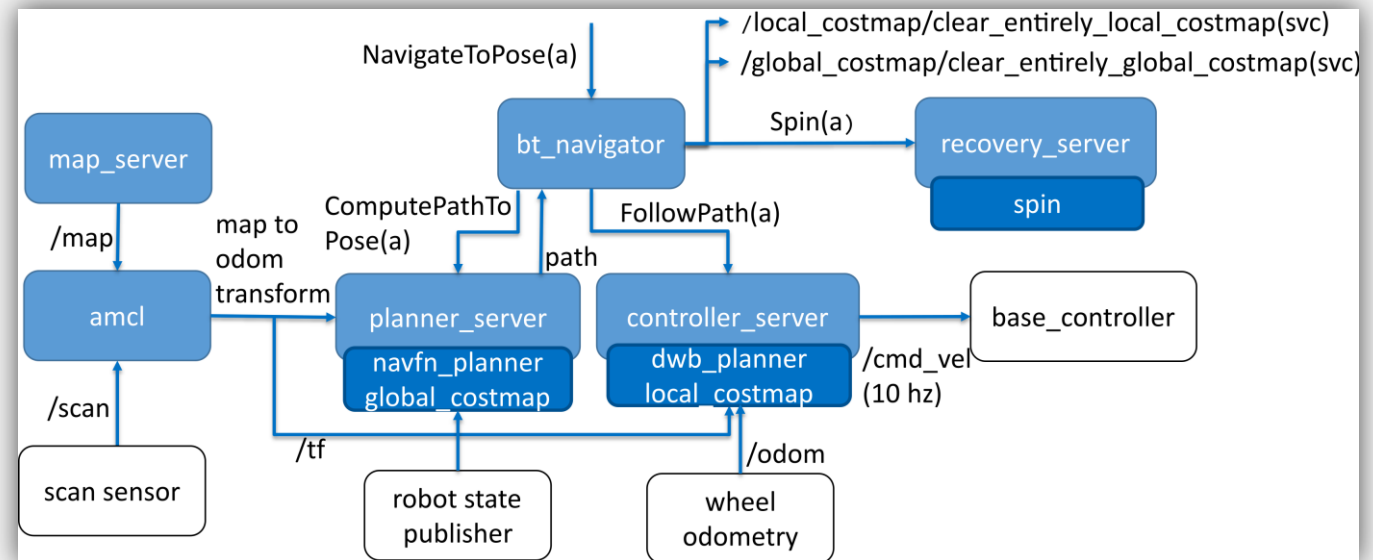


Diagrama de funcionamento do Nav2

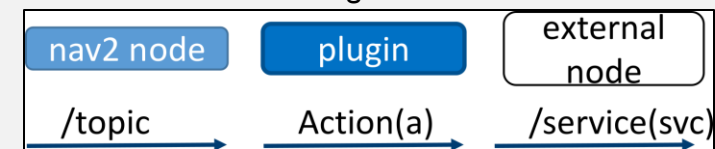
NAV2: CONCEITOS

- Nav2 retorna como saída:
 1. Comandos de velocidade para os motores do robô. Tipos de robôs móveis suportados:
 1. Holonômicos
 2. *Drive* diferencial
 3. Ackermann (tipo carro)
 4. Robôs com pernas
 2. Resultados de cada servidor individualmente

Funcionamento do Nav2 em termos da API do ROS

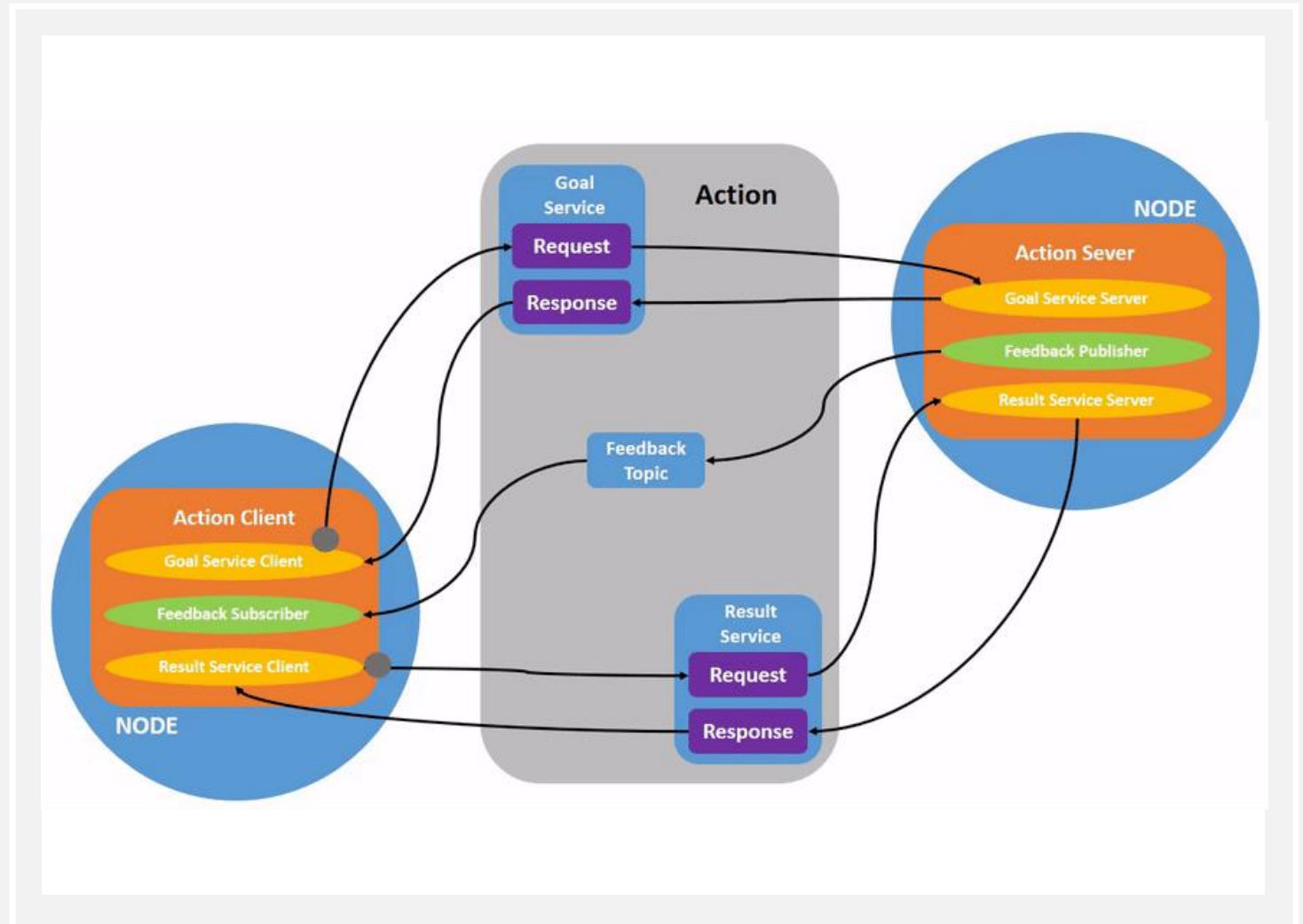


Legenda:



CONCEITOS: AÇÕES

- Comunicação via **ações**
 - Como “um serviço mais demorado”, com respostas periódicas
 - Uma ação contém 1 serviço de objetivo, 1 serviço de resultado, e 1 tópico de *Feedback* periódico
 - Exemplo: Mover um braço robótico para um ponto
 - Feedback periódico sobre onde ele está e o que está fazendo



CONCEITOS: SERVIDORES DE NAVEGAÇÃO



Planner

Calculam um caminho para completar um objetivo

Configurado em runtime com aliás e tipos de algoritmos a serem usados

Expõe interface de ação, que chama o algoritmo de planejamento selecionado para levar o robô de um lugar a outro



Smoother

Recebe uma rota do planner e suaviza ela

Aumenta distância entre robô e obstáculos percebidos

Minimiza descontinuidades na rota e movimentos súbitos que o robô teria de fazer



Controller

Também conhecido como planner local

Busca seguir a rota computada pelo planner global em um horizonte finito

Calcula um esforço de controle (em velocidades dos atuadores) para tentar seguir a rota vinda do suavizador



Behavior

Seleciona que comportamento o robô tem de realizar em um dado instante

Principal utilidade é ter "planos de emergência" para caso algo não funcione na rota planejada

Ex: uma porta que deveria estar aberta está fechada; servidor então vai para o comportamento "abrir porta"



Route

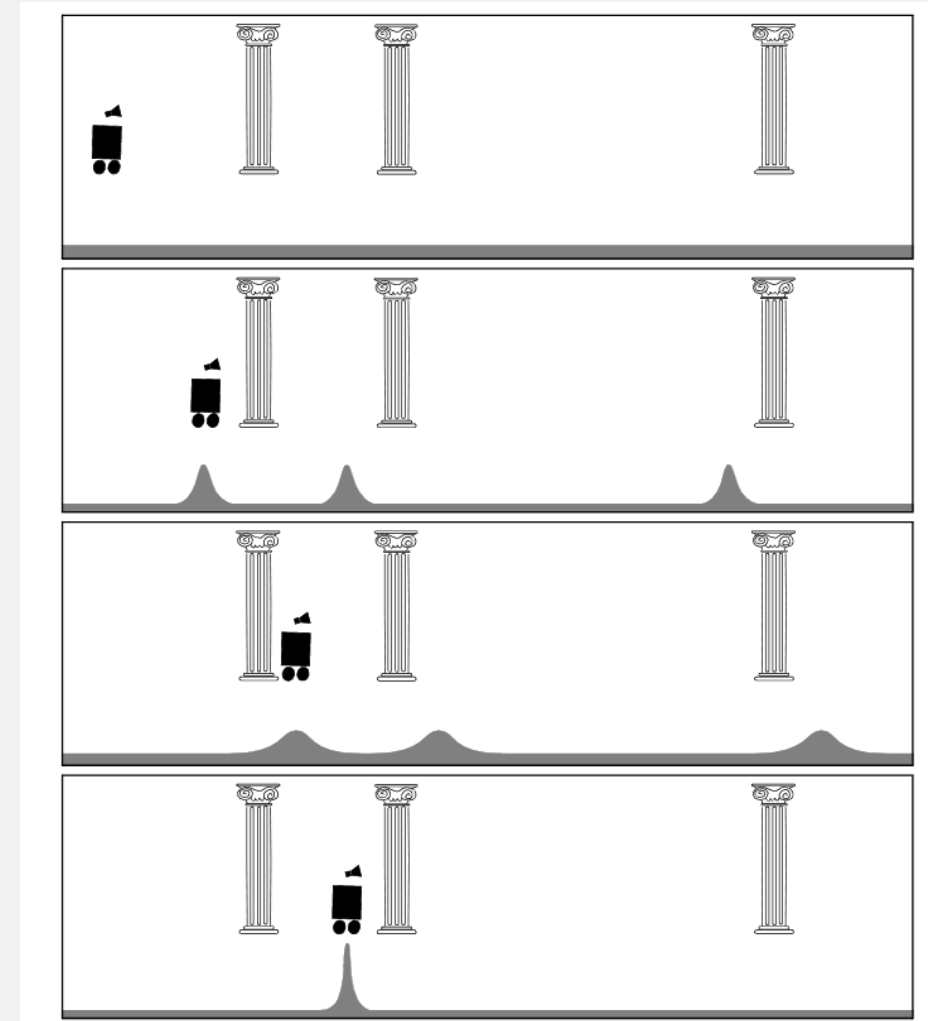
Planner especializado para computar uma rota usando costmaps, ao invés de um costmap de espaço livre

Calcula caminho ótimo de A para B pelo grafo de navegação

Grafo pode ser usado para generalizar mapa do ambiente

CONCEITOS: ESTIMAÇÃO DE ESTADOS

- Duas transformações principais devem ser fornecidas para o Nav2:
 - `\map` → `\odom`
 - Posição absoluta do robô no mapa
 - Fornecido por um sistema de posicionamento (i.e., localização, mapeamento, SLAM)
 - Nav2 providencia implementação `amcl` (Adaptive Monte-Carlo Localization)
 - `\odom` → `\base_link`
 - Posição relativa do robô baseada em odometria
 - Estimativa baseada em LIDAR, RADAR, Imus, Encoders, etc.
- Os dois são fundidos para estimar a posição do robô no mapa global

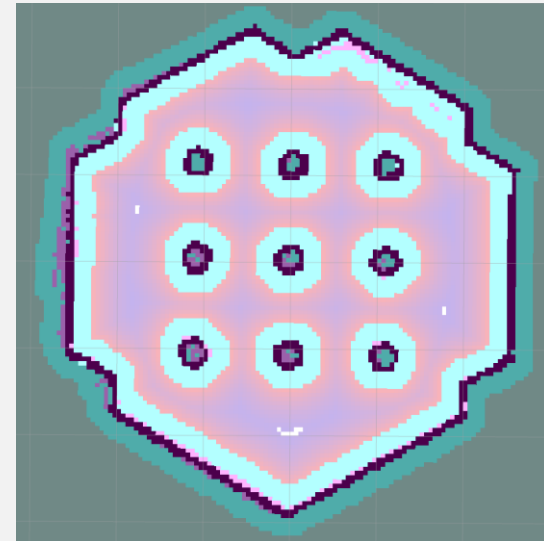


Fusão de dados de posicionamento absoluto e diferencial:
A crença de posições possíveis do robô entre medidas de posições absolutas (i.e., o robô avista uma coluna) se torna mais incerta com o tempo, até ele avistar outra *landmark*.
Figura retirada de [Siegwart, Nourbakhsh, "Introduction to Autonomous Mobile Robots", 2004]

CONCEITOS: REPRESENTAÇÃO DO AMBIENTE

Mapas no Nav2 são representados por mapas estáticos (*maps*) e mapas de custo (*costmaps*)

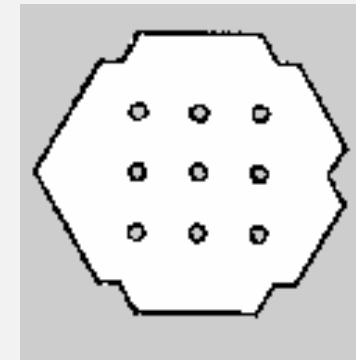
- Malha regular 2D de células contendo um *custo* de posição do robô estar em cada célula
 - 4 níveis: “desconhecido”, “livre”, “ocupado” e “inflado”
 - Custo é “inflado” perto de obstáculos, para evitar colisão
 - Mapa é então buscado (pensem nos algoritmos A*, D*) para planejar um rota que o robô deve seguir
- Mudanças no mapa são aplicadas como camadas
 - Ex: zonas restritas, caminhos preferidos, etc.
- Implementação: dois arquivos representam o mapa estático
 - `meu_mapa.yaml`: Dados sobre o mapa; resolução (metros/pixel), limites de probabilidade de ocupação
 - `meu_mapa.pgm`: Representação em escala de cinza do ambiente estático do mundo



Exemplo de mapa de custos gerado pelo planejador global, visto no Nav2

```
image: meu_mapa.pgm
mode: trinary
resolution: 0.050
origin: [-10.000, -10.000, 0]
negate: 0
occupied_thresh: 0.65
free_thresh: 0.196
```

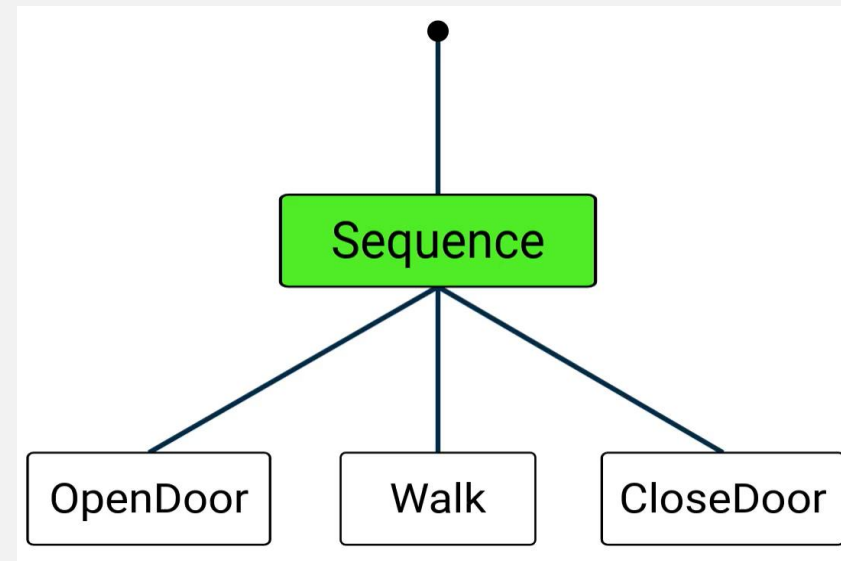
Arquivo `meu_mapa.yaml`

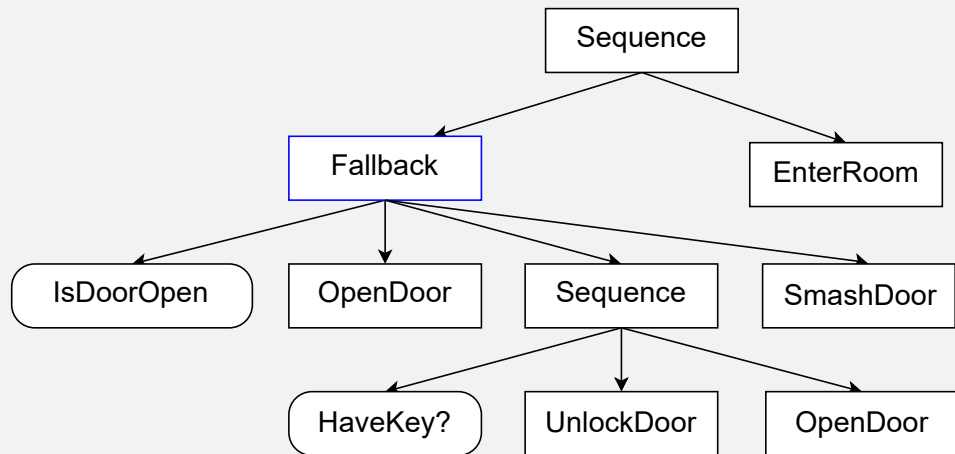
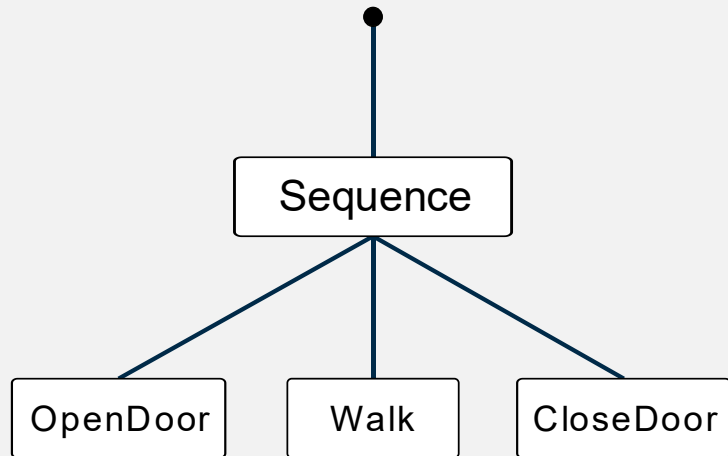


Arquivo `meu_mapa.pgm`

CONCEITOS: BEHAVIOR TREES

- Árvores de tomada de decisão (Behavior Trees (BT)):
 - É o algoritmo macro que liga uma sequência de ações com seus gatilhos
 - Outra maneira de modelar um problema, diferente da abordagem de máquinas de estado
 - No geral são mais apropriadas para atividades com muitos possíveis estados e transições, como navegação
 - Maior reatividade e capacidade de reutilização
- Toda iteração (tick) da árvore começa pela "base"
- Existem dois tipos de nós em uma BT
 - Controle (Não, não é o controle que aprendemos)
 - Comandam o fluxo lógico dos nós de ação
 - Ação
 - Executam tarefas ou verificam condições





BEHAVIOR TREES

- Árvores de tomada de decisão (Behavior Trees (BT):
 - Nós de "controle" possuem três tipos
 - Nós de sequência (Lógica AND) (?)
 - Executa os filhos em ordem. Se um falhar, a sequência inteira falha e para
 - Nós de Fallback/Selector (Lógica OR) (->)
 - Tenta o primeiro filho. Se ele falhar, tenta o segundo, e assim por diante. Só falha se *todos* falharem.
 - Nós paralelos
 - Roda vários filhos ao mesmo tempo. Pode esperar todos terminarem ou apenas um.

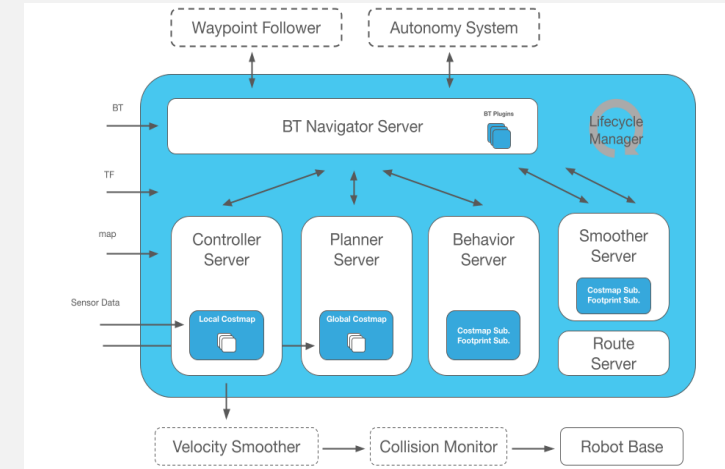
NAVEGAÇÃO NO ROS2

- Onde está a mágica:

```
planner_server:
  ros__parameters:
    # dois planejadores
    planner_plugins: ["Lento", "Rapido"]

    # Config Lento
    Lento:
      plugin: "nav2_navfn_planner/NavFnPlanner" # A*
      tolerance: 0.0

    # Config rapido
    Rapido:
      plugin: "nav2_navfn_planner/NavFnPlanner"
      tolerance: 2.0
```



[Navigation Plugins — Nav2 1.0.0 documentation](#)

BEHAVIOR TREES

- No ROS2: Implementado por [BehaviorTree.CPP v4](#)
 - Os nós de ação (da árvore) são nós (do ROS) que implementam ações
 - Retornam Running no feedback topic até ter um resultado
 - Nós de controle são implementados em mais baixo nível, diretamente em classes pelo *BT Navigator Server* e não seguem a mesma arquitetura que estamos acostumados no ROS

